

Prerequisite: ECA10

Course Objectives:

Students will:

1. Understand Embedded system design and analysis.
2. Develop knowledge of embedded programming concepts.
3. Explore the principles and functionalities of real-time operating systems (RTOS).
4. Make use of the architecture and programming of ARM processors.
5. Identify the characteristics and real-world applications of FPGAs, ASICs and DS Processors in various domains.
6. Gain insights into the fundamentals of SoC design.

Course Outcomes:

Upon successful completion of the course, students will be able to:

1. Apply formal design methodologies to develop and evaluate controller-based solutions for real-time applications.
2. Implement and assess advanced Embedded C programming techniques to design efficient embedded systems for real-world applications.
3. Design and analyze real-time operating system (RTOS) solutions, implementing efficient scheduling algorithms for robust and reliable embedded system performance.
4. Develop embedded systems with optimized memory and power utilization using hardware-software co-design techniques and optimization strategies.
5. Evaluate and design advanced architectures and communication interfaces to enhance system performance.
6. Model and create embedded applications using advanced design techniques.
7. Utilize embedded system development tools to simulate, analyze, and validate the functionality of real-time embedded systems.
8. Develop a fully functional prototype using advanced processors.

SYLLABUS

Unit I: EMBEDDED COMPUTING

1. Introduction

- Definition and scope of embedded systems
- Evolution of embedded systems (from simple controllers to complex SoCs)
- Applications in consumer, industrial, automotive, and healthcare domains
- Embedded systems in Industry 4.0 and 5.0 context

2. Complex Systems and Microprocessor

- Definition of complex embedded systems
- Features of complex systems
- Overview of microprocessors vs. microcontrollers
- CPU architecture basics: ALU, registers, control unit
- Embedded system performance factors (speed, power, memory, integration)

3. Embedded System Design Process

- Requirement analysis and specification
- Hardware–software partitioning
- Component selection (processors, memory, sensors, actuators)
- Verification, validation, and testing

4. Core Components

- Processing unit (MCU, MPU, DSP, SoC)
- Memory hierarchy (RAM, ROM, Flash, EEPROM)
- Input/Output interfaces (GPIO, ADC/DAC, serial communication)
- Sensors and actuators
- Power supply and management systems

5. Formalism for System Design

- State machine models (FSM, Mealy, Moore)
- Data flow models
- Petri nets for concurrency modeling
- UML and SysML in embedded systems
- Role of formal verification

6. Embedded Product Development Cycle

- Concept phase: Idea generation and feasibility analysis
- Design phase: Hardware and firmware development
- Implementation phase: Prototyping and integration
- Testing and validation: Functional, stress, and compliance tests
- Deployment, maintenance, and lifecycle management

7. Design Examples

- Model train controller system
- GPS-enabled tracking device

8. Networked Embedded Bus Protocols

- Overview of embedded communication standards
- Controller Area Network (CAN) protocol fundamentals
- Real-time communication requirements in embedded networks

9. CAN Bus and Extended Protocols

- CAN bus frame structure (standard vs. extended identifiers)
- Bit timing and synchronization
- Error handling and arbitration
- CAN FD (Flexible Data-rate):
 - Differences from classic CAN

- Increased payload size and higher data rates
- Applications in automotive and industrial systems

10. Smart Home Automation

- Embedded controllers for home automation
- Sensor and actuator control for lighting, HVAC, security

11. Predictive Health Assessment System

- Role of embedded systems in medical monitoring
- Sensors for vital signs: ECG, SpO₂, temperature, motion
- Wireless health data transmission and security considerations

12. CAN Architecture in Luxury Automobiles

- Distributed control modules in vehicles (ECUs)
- CAN-based communication for infotainment, engine, braking, and safety systems
- Fault tolerance and redundancy in luxury automotive systems

UNIT II :EMBEDDED C PROGRAMMING

1. Features of Embedded Programming Languages

- Overview of embedded language requirements (efficiency, portability, determinism)
- Low-level hardware interaction capabilities
- Real-time performance constraints
- Memory and resource optimization features

2. Difference between C and Embedded C

- Standard C (ANSI C) scope vs. Embedded C scope
- Compiler and toolchain differences
- Use of hardware-specific libraries and register-level programming

3. Key Characteristics of Embedded C

- Deterministic execution and real-time response
- Portability across microcontroller platforms
- Direct hardware control via pointers and registers
- Low memory footprint and power optimization
- Integration with assembly language for performance

4. Standard Embedded C Data Types

- Integer types (int8_t, int16_t, int32_t, uint8_t, etc.)
- Floating-point types and their usage constraints
- Boolean and bitwise operations
- Enumerations and typedefs for code readability

5. Block Diagram Explanation of Embedded C

- Hardware layer (microcontroller, peripherals)
- Low-level drivers and HAL (Hardware Abstraction Layer)
- Application layer logic
- Interaction between code modules, interrupts, and peripherals
- Example diagram for a simple sensor-reading application

6. Basic Embedded C Programming Steps

- Problem definition and requirement analysis
- Writing pseudocode and flowcharts
- Setting up development environment (IDE, compiler, debugger)
- Writing, compiling, and linking code
- Flashing firmware to target hardware
- Testing and debugging

7. Vibration Monitoring Node Design

- Selection of microcontroller and vibration sensor (e.g., accelerometer)

- ADC configuration for vibration data acquisition
- Implementing sampling and filtering in Embedded C
- Threshold detection and alert mechanism

8. Industrial HVAC Control Firmware Design

- Overview of HVAC system control requirements
- Sensor interfacing (temperature, humidity, airflow)
- Actuator control (fans, compressors, dampers)
- Control algorithm implementation (PID, ON-OFF control)
- Safety features and fault handling in firmware

UNIT III: EMBEDDED C APPLICATIONS

1. Adding Structure to 'C' Code

- Benefits of structured programming in embedded systems
- Applying Object-Oriented Programming (OOP) principles in C (encapsulation via struct, function pointers for polymorphism, modular design)
- Organizing large projects using header files (.h) and source files (.c)
- Creating reusable libraries for embedded projects

2. Meeting Real-Time Constraints

- Real-time system requirements and deadlines
- **Creating hardware delays:** Using timers and counters
- **Need for timeout mechanisms:** Avoiding infinite loops and deadlocks
- **Creating loop timeouts:** Software-based time checks
- **Creating hardware timeouts:** Using watchdog timers and hardware peripherals

3. Application Development using SystemC

- Overview of SystemC for hardware–software co-design
- Modules, ports, and signals in SystemC
- Modeling concurrency and timing
- Simulation and verification in SystemC environment

4. Python Scripting for Embedded Test Automation

- Introduction to Python in embedded workflows
- Automating serial communication tests (PySerial)
- Automating sensor data logging and analysis
- Integration with CI/CD pipelines for firmware testing

5. Using Python for Firmware Flashing and Debugging Tools

- Python libraries for hardware programming (PyOCD, esptool.py)
- Automating firmware flashing to microcontrollers
- Controlling JTAG/SWD debuggers with Python scripts

6. SystemC-Based Production Line Simulator

- Concept of digital twins for industrial systems
- Modeling sensors, actuators, and conveyor systems in SystemC
- Simulating faults and maintenance events
- Linking SystemC models with external dashboards or scripts

7. Embedded C Predictive Maintenance Dashboard Design

- Vibration, temperature, and current sensing for maintenance
- Embedded C data acquisition and preprocessing
- Sending data to cloud dashboards (MQTT, HTTP)
- Visualizing trends for anomaly detection

8. Real-Time Systems in Smart Cities

- Role of embedded systems in smart traffic, energy, and public safety
- Integration with IoT and edge computing

- Real-time monitoring and decision-making algorithms
- Case studies: Smart traffic light control, environmental monitoring
- Example: Real-time water level monitoring for flood alerts

UNIT- IV: REAL TIME OPERATING SYSTEMS

1. Principles of RTOS

- Definition and characteristics of real-time systems
- RTOS vs. general-purpose OS
- Hard vs. soft real-time systems
- RTOS architecture: kernel, scheduler, interrupt handler, inter-task communication

2. Semaphores and Queues

- Purpose of semaphores in RTOS
- Binary, counting, and mutex semaphores
- Use cases for queues in task communication
- Differences between queues and semaphores

3. Task and Task States

- Definition of a task in RTOS
- Task control block (TCB) structure
- Task lifecycle: ready, running, blocked, suspended
- Task switching and context switching process

4. Tasks and Data, Semaphores, and Shared Data

- Problem of shared data in concurrent tasks
- Protecting shared resources using semaphores
- Avoiding race conditions and deadlocks
- Priority inversion and its impact on data handling

5. Message Queues, Mailboxes, and Pipes

- Concepts of message passing in RTOS
- Message queues: fixed-size and variable-size messages
- Mailboxes: single message storage and retrieval
- Pipes: stream-based inter-task communication

6. Timer Functions and Events

- Hardware timers vs. software timers
- Periodic and one-shot timers
- Event flags and event groups in RTOS

7. Memory Management in RTOS

- Static vs. dynamic memory allocation
- Memory fragmentation issues in real-time systems
- Memory pools and fixed block allocation

8. Interrupt Routines in RTOS Environment

- Role of ISRs (Interrupt Service Routines) in real-time systems
- ISR latency and minimization techniques
- Interaction between ISRs and tasks

9. Real-Time Task Scheduling and Scheduling Algorithms

- Fixed-priority scheduling
- Rate Monotonic Scheduling (RMS) – theory and examples
- Earliest Deadline First (EDF) – theory and examples
- Scheduling analysis for feasibility checking

10. Resource Sharing and Priority Inheritance Protocols

- Resource sharing issues in multitasking
- Priority inversion problem

- Priority inheritance and priority ceiling protocols

11. RTOS-Driven Autonomous AGV Controller Design

- Overview of AGV navigation and control requirements
- Task breakdown: navigation, obstacle detection, motor control
- Communication between RTOS tasks in AGV systems
- Integration of sensors, actuators, and decision-making logic

12. Real-Time Robotic Cell Task Scheduler

- Overview of robotic cell operations in manufacturing
- Task scheduling for multiple robots and conveyors
- Handling synchronization between robots and machines
- Implementation of RMS/EDF in robotic cells

UNIT–V: ADVANCED ARCHITECTURES AND APPLICATION DEVELOPMENT

1. Introduction to SoC Devices and Platforms

- Definition and architecture of System-on-Chip (SoC)
- Selection criteria for SoC in embedded applications
- Overview of supported peripherals, memory, and connectivity features
- Development toolchains and ecosystems for SoC devices

2. ESP and ESP32 Platform

- Architecture and features of ESP32 (Wi-Fi + Bluetooth)
- GPIO, ADC/DAC, PWM, and communication interfaces
- Firmware development using Arduino IDE, ESP-IDF, and PlatformIO

3. Teensy 4.1 Platform

- Hardware architecture and high-speed performance capabilities
- Peripheral interfacing (USB host, CAN bus, Ethernet)
- Real-time audio processing applications

4. Odroid and UDOO Boards

- Odroid: ARM-based SBC architecture, multimedia and AI capabilities
- UDOO: Hybrid ARM + x86 processing, GPIO expansion, Arduino compatibility
- High-performance computing for robotics and automation

5. TinyML and Edge AI

- Introduction to TinyML concepts and frameworks (TensorFlow Lite Micro, Edge Impulse)
- Model quantization, compression, and deployment on microcontrollers
- Case study: Edge AI vision recognition gateway design

6. Edge AI Vision Recognition Gateway Design

- Hardware selection for vision-based applications
- Image capture, preprocessing, and feature extraction
- Integrating ML models for inference at the edge
- Communication with cloud-based AI services

7. Cognitive Control Engine Design

- Concept of cognitive control in embedded systems
- Adaptive decision-making algorithms
- Sensor fusion for enhanced situational awareness
- Application in robotics and industrial automation

8. Self-Healing Embedded Network Design

- Principles of self-healing networks in IoT/embedded environments
- Fault detection and recovery mechanisms
- Network redundancy and failover strategies

9. Debugging a Communication Failure using a Logic Analyzer

- Logic analyzer fundamentals and operation
- Setting up probes for bus communication monitoring
- Protocol decoding for UART, SPI, I²C, CAN
- Identifying and resolving communication faults

10. Managing Source Code and Schematics using Git

- Git basics: repository creation, branching, and merging
- Version control for firmware, schematics, and PCB layouts
- Collaborating in embedded hardware–software teams

11. Debugging using JTAG

- JTAG protocol basics and IEEE 1149.1 standard
- JTAG hardware setup and connection to target devices
- Using JTAG for firmware flashing, breakpoints, and memory inspection

SYLLABUS

UNIT-I: EMBEDDED COMPUTING

9

Introduction, Complex Systems and Microprocessor, Embedded System Design Process, Core components, Formalism for System Design, Embedded product development cycle Design Examples. **Networked embedded Bus protocols, CAN bus and extended protocols - CAN FD, Smart Home Automation, Predictive Health Assessment System, CAN architecture in Luxury Automobiles**

UNIT – II: EMBEDDED C PROGRAMMING

9

Embedded C programming: Features of embedded programming languages, Difference between C and Embedded C, Key Characteristics of Embedded C, Standard Embedded C Data Types, Block Diagram Explanation of Embedded C, Basic Embedded C Programming Steps, **Vibration Monitoring Node design, Industrial HVAC Control Firmware design**

UNIT – III EMBEDDED C APPLICATIONS

9

Adding Structure to ‘C’ Code: Object oriented programming with C, Header files for Project and Port, Examples. Meeting Real-time constraints: Creating hardware delays - Need for timeout mechanism -Creating loop timeouts - Creating hardware timeouts, Application development using SystemC, Python scripting for embedded test automation, Using Python for firmware flashing and debugging tools, SystemC-Based Production Line Simulator, Embedded C Predictive Maintenance Dashboard design, Real-Time systems in Smart Cities

UNIT- IV: REAL TIME OPERATING SYSTEMS

9

Principles, Semaphores and Queues, Task and Task States, Tasks and Data, Semaphores, and Shared Data; Message Queues, Mailboxes and Pipes, Timer Functions, Events, Memory Management, Interrupt Routines in RTOS Environment. Real-time task scheduling, and scheduling algorithms (RMS, EDF). Resource sharing and priority inheritance protocols, **RTOS-Driven Autonomous Automated Guided Vehicles (AGV) Controller design, Real-Time Robotic Cell Task Scheduler.**

UNIT-V: ADVANCED ARCHITECTURES AND APPLICATION DEVELOPMENT

9

Design and development of practical application using SoC devices -ESP - ESP32 Platform, Teensy 4.1, Odroid, UDOO, TinyML, Edge AI Vision Recognition Gateway design, Cognitive Control Engine design, Self-Healing Embedded Network design, Debugging a communication failure using a logic analyzer, Managing source code, schematics using GIT, Debugging using JTAG

Text Books

1. Wayne Wolf, “Computers as Components-Principles of Embedded Computer System Design”, Elsevier, 2008.
2. Kenneth J. Ayala, “The 8051 Microcontroller”, Third Edition, Thomson, 2004.

References

1. Embedding system building blocks, Labrosse, via CMP publishers
2. Embedded Systems, Raj Kamal, TMH
3. Micro Controllers, Ajay V Deshmukhi, TMH
4. Embedded System Design, Frank Vahid, Tony Givargis, John Wiley
5. Microcontrollers, Raj kamal, Pearson Education
6. An Embedded Software Primer, David E. Simon, Pearson Education.
7. Raj Kamal, "Hands-ON Embedded Program. T", ISBN 13-978-9353168025, ISBN 10-9353168.
8. Yifeng Zhu, "Embedded System with ARM cortex-M". ISBN 13-978 0982692660 and ISBN 10-0982692668.
9. J.W. Valvano, "Embedded Microcomputer System: Real Time Interfacing", Brooks/Cole, 2000.
10. Jack Ganssle, "The Art of Designing Embedded Systems", Newnes, 1999.
11. V.K. Madisetti, "VLSI Digital Signal Processing", IEEE Press (NY, USA), 1995.
12. K.J. Ayala, "The 8051 Microcontroller: Architecture, Programming, and Applications", Penram Intl, 1996.
13. ARM System-on-Chip Architecture by Steve B. Furber
14. Michael J. Pont, “Embedded C”, Addison-Wesley Professional Publisher 2002
15. M.J.S .Smith, "Application Specific Integrated Circuits” Pearson Education , 2004.